PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text-to-Cypher Systems

Anonymous ACL submission

Abstract

Enterprises increasingly use property graphs and Cypher to analyze fraud, risk, identity, customer, and operational data. A useful enterprise Text2Cypher benchmark must therefore be private, refreshable, executable, and sensitive to graph-specific semantics such as relationship direction, literal grounding, and schema vocabulary. We present PIPE-Cypher, a local-model pipeline for generating organization-specific natural-language-to-Cypher benchmarks. PIPE-Cypher combines schema introspection, reverse-query grounding, constrained generation, deterministic Cypher governance, execution validation, diversity controls, and LLM-judge review. In live runs with local Qwen3.5-9B generation and judging, PIPE-Cypher exports 3,000 accepted FinBench/SNB examples, completes three audited ablation suites, calibrates an automated judge, onboards ICIJ Offshore Leaks, and evaluates 12 local downstream models. The resulting benchmark is difficult zero-shot, and leakage-aware few-shot controls show that schema-specific examples can substantially improve compatible model families while still exposing brittle transfer failures.

1 Introduction

Property graphs are attractive in enterprise settings because the facts of interest are often relational paths: account transfers, identity entitlements, access chains, customer interactions, supplier dependencies, and fraud rings. Cypher gives analysts a compact language for these patterns. As LLMs become natural-language interfaces for graph analytics, an organization cannot evaluate a Text2Cypher system only on public schemas; it needs to know whether the model handles its labels, relationship directions, values, governance rules, and recurring operational questions.

This changes the benchmark problem. A static public dataset is valuable for shared comparison, but it cannot contain a bank's account taxonomy, an identity team's permission graph, or the exact categorical values that make a query answerable. It also cannot refresh when schemas change. Industry teams therefore need a benchmark factory: a repeatable way to turn a live property graph into a balanced, executable, privacy-aware NL-to-Cypher benchmark without sending sensitive schema or values to paid generation APIs.

PIPE-Cypher addresses this setting. The pipeline profiles a target graph, grounds question slots through reverse Cypher execution, constrains local-model generation, validates and repairs generated Cypher through deterministic gates, and uses a local LLM judge only after execution evidence is available. Its design treats Cypher correctness as a governed artifact rather than a prompt preference: relationship direction, read-only safety, exact literal use, categorical values, contextual return columns, and `RETURN DISTINCT` are checked or normalized before examples are accepted.

Contributions. We make four contributions: (1) an end-to-end local-model pipeline for private enterprise NL-to-Cypher benchmark generation; (2) a Cypher governance layer that turns production query-writing constraints into deterministic validation and conservative rewrite rules; (3) a scaled evaluation over FinBench, SNB, and ICIJ showing balanced generation, ablations, judge calibration, and a 12-model local transfer stress test; and (4) reproducible artifacts for enterprise onboarding, privacy redaction, value sampling, benchmark refresh, and appendix-level auditability.

2 Related Work

Recent Text2Cypher resources, including Mind the Query (Chauhan et al., 2025), SyntheT2C (Zhong et al., 2025), Auto-Cypher (Tiwari et al., 2025), and Text2Cypher (Ozsoy et al., 2025), show that high-quality Cypher data requires schema grounding, execution checks, verification, and complexity-aware evaluation. Mind the Query is especially relevant because it reports an Industry Track Text2Cypher dataset with schema, runtime, value, and human logical validation. PIPE-Cypher borrows the reviewer-facing discipline of those checks, but changes the target artifact: instead of releasing one static dataset, it studies the enterprise process of generating, refreshing, auditing, and redacting benchmarks for an organization's own graph, local model endpoint, privacy policy, and benchmark refresh cycle.

Text-to-SQL benchmarks such as Spider 2.0 (Lei et al., 2024) and BIRD (Li et al., 2023) have pushed text-to-query evaluation toward realistic database tasks and execution-based scoring. Recent residual-skill Text-to-SQL work further shows that optimizing complementary agent skills on residual failures can improve selected accuracy across SQL dialects (Zhu et al., 2026). Graph query generation adds different failure modes: relationship direction can invert the meaning of a query, node and relationship properties live in different namespaces, and path-shaped questions can be syntactically valid while semantically ungrounded. CIKM AutoQuery (Zheng et al., 2024) motivates treating workload generation as a separate object of study from downstream model quality. LDBC FinBench (Qi et al., 2023) and SNB (P üroja et al., 2023) provide public graph workloads with financial and social-network structure, while ICIJ Offshore Leaks gives an additional public finance/compliance onboarding check.

For benchmark quality, lexical diversity alone is not enough. PIPE-Cypher combines text-generation diagnostics such as Distinct-n (Li et al., 2016) and self-BLEU-style redundancy checks (Zhu et al., 2018) with text-to-query structure metrics: schema coverage, relationship/property coverage, query-signature diversity, structural feature rates, and normalized entropy over graph/category/difficulty cells. These metrics make a practical distinction that matters to enterprise users: a benchmark can be syntactically diverse while still overusing the same values or query templates.

3 Method

PIPE-Cypher has six stages: schema profiling, workload planning, reverse Cypher grounding, constrained Cypher generation and repair, deterministic validation and execution, and LLM-judge review. The central design choice is to make answerability and governance executable. Prompts can ask a model to respect relationship directions or exact literals, but accepted examples must prove those properties through schema checks, parser-style structure extraction, live execution, and judge review.

Schema profiling records labels, relationship types, properties, observed directions, and bounded low-cardinality categorical values. Workload planning targets eight operational categories: simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k. Reverse grounding runs read-only Cypher to bind slots to values that actually produce rows, reducing the common synthetic-data failure where a plausible question has no answer in the graph.

The deterministic layer enforces read-only safety, syntax shape, schema-visible labels and properties, explicit relationship types, observed relationship directions, schema-provided categorical values, and non-empty execution where required. A lightweight Cypher analyzer extracts return aliases, variables, labels, relationship observations, risky constructs, and rewrite skip reasons so normalization is auditable rather than a silent string edit. The direction gate interprets both outgoing and incoming Cypher arrow syntax before schema checking, and rejects undirected relationship patterns so directionality is an executable constraint rather than only a prompt instruction.

4 Implementation

The implementation is a Python package with Neo4j as the experimental backend. The paper frames the contribution around Cypher and property graphs rather than a single vendor. Reported benchmark generation and judge review use a local Qwen3.5-9B endpoint behind a vLLM/OpenAI-compatible interface. Downstream evaluation separately tests 12 locally served model checkpoints spanning general instruction, code-tuned, Cypher-tuned, and Text2Cypher-tuned families. This keeps the study aligned with an enterprise deployment pattern in which benchmark generation and evaluation run inside the organization's compute boundary without paid generation APIs.

The built-in FinBench profile is grounded in the

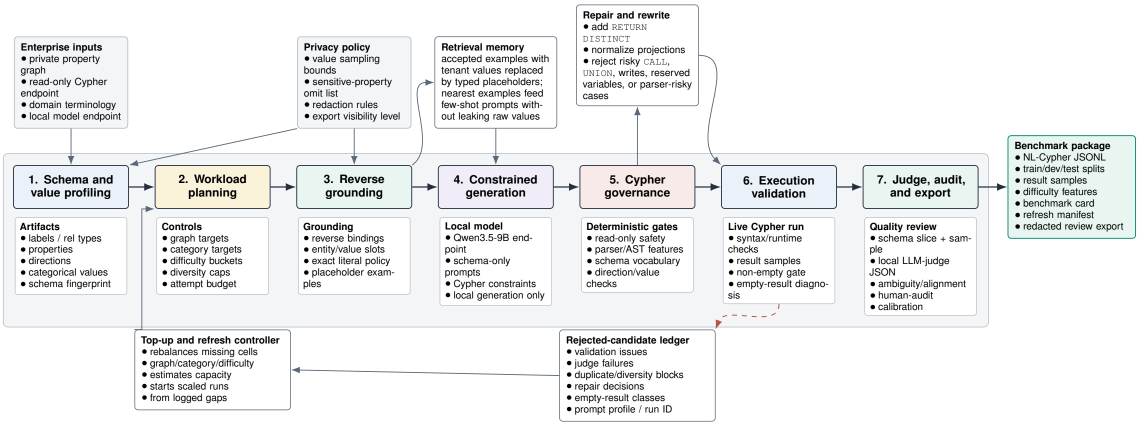

Figure 1: PIPE-Cypher benchmark generation pipeline. The key industry additions beyond static Text2Cypher dataset construction are privacy/value policies, reverse grounding, Cypher governance, execution diagnostics, local judge calibration, and benchmark export/refresh.

Gate	Evidence checked	Failure mode caught
Read-only safety	blocked Cypher write/admin tokens	destructive or operational queries
Schema validity	labels, relationship types, properties, categorical values	hallucinated graph vocabulary or values
Direction validity	observed relationship directions	reversed or invalid traversals
Cypher analysis and normalization	structure extraction, rewrite safety, `RETURN DISTINCT`	unsafe rewrites or duplicate/noisy rows
Question constraints	quoted values use exact literals	fuzzy matching of exact user values
Execution	query runs and returns rows	syntactic validity without answerability
LLM judge	ambiguity, semantic alignment, schema use	executable but semantically weak examples

Table 1: PIPE-Cypher candidate acceptance gates.

public datagen snapshot export and includes typed node and relationship properties. Live schema inspection also records low-cardinality string values as categorical constraints for prompting and validation. The generated Neo4j import script uses uniqueness constraints for nodes and creates, rather than merges, transaction relationships so repeated account-to-account events are preserved. The SNB reference profile is grounded in the official Neo4j/Cypher headers and read-query files.

For generation, PIPE-Cypher uses seeded workload templates with deterministic reverse-binding queries and a mixed mode that prepends proven workload seeds to LLM-proposed templates. Every run records accepted and rejected candidates for yield analysis. Retrieved few-shot examples replace graph-specific values with typed placeholders, preserving query structure while reducing tenant-value leakage and memorized entity reuse. A dependency-light value grounder adds typed prompt annotations for categorical values and reverse-bound entities, covering punctuation variants, possessives, plurals, synonyms, name partials, and small typos.

Inspired by Mind the Query's prompt-setting analysis, the implementation exposes prompt profiles for schema-only, instructions-only, examples-only, examples-plus-instructions, and full governed PIPE-Cypher generation. These profiles are configured as ablation variants and must pass the same target-size and paper-readiness standards before any result is reported.

For LLM-judge review, the implementation slices schema context to the labels, relationship types, and properties used by the candidate query. This preserves validation against the full schema while keeping local 9B prompts within context limits on larger schemas.

The deterministic Cypher layer uses production-derived constraints: schema-only prompting, exact matching, relationship direction discipline, `RETURN DISTINCT`, reserved variable rejection, categorical values, required contextual return columns, fuzzy value annotations, placeholderized retrieval examples, and parser-aware rewrite boundaries. PIPE-Cypher records parser-style structure features and skips rewrites for risky constructs such as `UNION`, `CALL`, `UNWIND`, `WHERE EXISTS`,

Graph	Target/category	Binding limit	Min seed capacity	All categories meet target
FinBench	250	300	300	Yes
SNB	125	200	200	Yes

Table 2: Built-in seed-template capacity after removing the reverse-binding 10-row cap and adding slotted negation/ranking seeds. Capacity is a theoretical scale-readiness check; final examples still must pass validation, execution, diversity, and judge gates.

Setting	Value
Accepted examples	3,000
Primary graph	LDBC FinBench, 2,000 examples
Secondary graph	LDBC SNB, 1,000 examples
Categories	8 balanced categories, 375 each
Generation/judge model	Local Qwen3.5-9B
Execution backend	Neo4j Community, two live databases
Judge audit packet	80 sampled accepted/rejected pairs

Table 3: Full live experimental setup used for the generated benchmark artifact.

multiple `WHERE` clauses, or reserved variables.

We also estimate seed-template capacity before full generation. This check caught and fixed a scale blocker: reverse-binding execution was hard-capped to 10 rows, and no-slot negation/ranking seeds could not support full category targets. PIPE-Cypher now uses the configured binding limit and includes slotted negation and ranking seeds for both graphs.

For arbitrary enterprise onboarding, PIPE-Cypher includes a deployment template for a company's own graph, read-only credentials, local model endpoint, schema introspection, privacy policy, dry run, scaled run, audit, and redacted export. Configurable value-sampling policies bound which low-cardinality graph values enter schema prompts, and redacted exports replace quoted literals, entity values, and string-valued result samples with stable placeholders for broader internal review. The ICIJ onboarding run further motivated schema-derived sparse-category templates: PIPE-Cypher now derives relationship-count, anti-join, and top-k templates from observed labels, relationship directions, and safe low-cardinality properties, then grounds slot values with outcome-aware reverse Cypher.

5 Experiments

Research questions. We evaluate PIPE-Cypher around four questions that matter for an industry benchmark generator: RQ1, can a local-model pipeline produce a balanced executable benchmark over live property graphs? RQ2, do Cypher-specific governance and grounding steps make generation reliable at scale? RQ3, does the resulting benchmark expose meaningful downstream Text2Cypher failures rather than merely checking syntax? RQ4, can the same machinery onboard a new public enterprise-style graph without hard-coding FinBench or SNB?

The generated benchmark contains 3,000 accepted examples: 2,000 from LDBC FinBench and 1,000 from LDBC SNB. Categories include simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k. Appendix Table 11 reports graph statistics, including the audited ICIJ third-graph onboarding workload.

We report three completed ablation suites over FinBench and SNB: an initial target-50 suite, a corrected target-100 suite, and a seed-17 target-50 repeat. Each suite contains 14 graph/setting cells over unconstrained local generation, reverse-only generation, validators+repair, no retrieval, no rewrite, no LLM judge, and full PIPE-Cypher. All paper-reported ablations have collection manifests, model IDs, code revisions, run summaries, and readiness audits. We additionally report a live ICIJ Offshore Leaks onboarding run as public finance/compliance evidence for arbitrary-schema deployment beyond LDBC.

Downstream Text2Cypher evaluation uses execution accuracy and answer-set precision/recall/F1 as primary correctness metrics. The benchmark evaluator also supports supplementary reference-based text metrics over serialized answer sets and query strings, including ROUGE, BLEU, METEOR, BERTScore, FrugalScore, cosine similarity, Jaro-Winkler similarity, and exact match; Appendix Table 27 lists the supported metrics and citations. We treat these as debugging and near-match diagnostics rather than substitutes for executable correctness. Appendix Table 15 and Figures 9–10 further separate workload-category balance from Cypher operator-strategy coverage.

6 Results

RQ1: executable benchmark generation. The full live run produced 3,000 accepted examples from 4,777 candidates using local Qwen3.5-9B for generation and judging. Category-specific recovery top-ups filled the only under-target categories from the initial sequential run. Every exported example passed read-only, syntax, schema, execution, non-

Graph	Candidates	Accepted	Acceptance	Categories at target
FinBench	3,404	2,000	0.588	8/8
SNB	1,373	1,000	0.728	8/8
Total	4,777	3,000	0.628	16/16

Table 4: Full live generation with local Qwen3.5-9B. Candidate counts include the initial sequential run and category-specific recovery top-ups.

Export artifact	Examples	FinBench	SNB	Train	Dev	Test
Live full benchmark	3,000	2,000	1,000	2,408	296	296

Table 5: Accepted live full benchmark package with stable IDs, gate metadata, result samples, statistics, and manifest hash `8bc79a53a06b291a`.

Artifact property	Value
Categories	8 balanced categories
FinBench/category	250
SNB/category	125
Difficulty split	1,521 easy / 1,479 medium
Used labels / rel. types	7 / 9
Read/syntax/schema/exec/judge gates	3,000/3,000

Table 6: Distribution and gate summary for the exported full benchmark artifact.

empty result, and judge gates.

The accepted full-run records were exported into a benchmark package with stable example identifiers, train/dev/test splits, result samples, gate metadata, aggregate statistics, and a manifest hash for artifact tracking. The export contains 3,000 accepted examples: 2,000 FinBench, 1,000 SNB, and 375 accepted examples in every planned category.

Diversity and residual concentration. We report diversity as an audit signal, not as a slogan. The full export has perfect category balance and near-perfect difficulty balance, uses 1,115 unique grounded entity values, and exactly quotes grounded values in 82.6% of examples with entity bindings. The reported local-model run still has low query-signature diversity because seeded graph-grounded templates are doing substantial work. This supports the intended industry workflow: PIPE-Cypher can produce a balanced executable benchmark, and it also exposes the residual template and value concentration that a benchmark owner should monitor during refresh or fine-tuning data generation.

RQ2: governed generation. The full-run failure taxonomy is concentrated in diversity/duplicate controls during recovery and empty execution results; only 2/4,777 candidates were schema-invalid after deterministic Cypher governance. The scaled ablation suites should be read as a reliability study of the execution-grounded core rather than as a claim that every optional module independently increases yield. In the target-100 suite, every non-unconstrained graph/setting cell reached all eight category targets with 800 accepted examples per cell; the full PIPE-Cypher setting accepted 800/824 candidates on both FinBench and SNB. Figure 2 shows the contrast with unconstrained local generation, which can emit plausible simple queries but does not provide balanced, reliable coverage under executable benchmark gates. Across the three evidence-ready suites, target-normalized coverage is 1.000 for every non-unconstrained cell; component value is therefore clearest in gate diagnostics, failure taxonomy, and auditability, while reverse grounding plus deterministic governance forms the robust minimum viable core.

Judge calibration. The released artifacts include an 80-row audit packet sampled from accepted and rejected full-run candidates, with a 40/40 judge accept/reject split and coverage over both graphs and all eight categories. A completed human audit shows 80.0% agreement, Cohen's $\kappa = 0.60$, judge precision and specificity of 1.00, recall of 0.714, and no false accepts in the sample. The judge is therefore conservative: it protects accepted-example quality while rejecting some human-approved candidates and reducing yield. Human labels are used only for post-hoc calibration, not as a generation gate.

RQ3: downstream stress test. We ran downstream Text2Cypher evaluation on the exported full test split. Each model saw schema text and the natural-language question, generated Cypher, and was evaluated by live execution against the corresponding FinBench or SNB database. The purpose is not to present a single baseline as strong; it is to test whether PIPE-Cypher exposes failures in plausible Text2Cypher outputs. A 12-model local transfer suite confirms this role: zero-shot execution accuracy ranges from 0.000 to 0.291, with a mean of 0.139. A stricter scored control that excludes exact query-signature matches and near-duplicate questions raises mean accuracy to 0.245, while ordered and random same-category example-bank controls reach 0.380 and 0.378 (Appendix Tables 8–9). Model-level bootstrap intervals show that gains are concentrated in compatible model families rather than universal. Public Text2Cypher-style tuning therefore does not automatically transfer zero-shot to enterprise-style graphs, while ac-

Split	Examples	Parse	Schema	Exec. success	Exec. acc.	Answer F1
Live full test	296	0.959	0.905	0.622	0.189	0.189

Table 7: Downstream Text2Cypher evaluation for local Qwen3.5-9B on the exported full benchmark test split.

cepted graph-specific examples provide an immediate operational artifact: a schema- and workload-specific question-answer bank for retrieval, adaptation, and future tenant-specific training data creation.

RQ4: third-graph onboarding. ICIJ onboarding reaches the same per-category target on a larger public finance/compliance graph: 800 accepted examples from 983 candidates over 2.0M nodes and 3.3M relationships, with 100 examples in every category. This result is important because it exercises schema-derived sparse-category templates and live reverse grounding on a graph whose schema is not one of the two LDBC study workloads. In practice, ICIJ forced the pipeline to derive relationship-count, anti-join, and top-k templates from observed schema structure instead of relying only on preauthored LDBC seeds.

7 Industry Use

PIPE-Cypher is intended to be rerun when an enterprise graph changes. The generated artifact records the schema snapshot, graph profile, model identifier, validation gates, execution samples, judge scores, difficulty features, and source run for every example. Long-running jobs can be monitored from JSONL records and recovered with category-specific top-up runs that reject questions accepted in earlier runs. This turns benchmark maintenance into an operational workflow: refresh after schema changes, compare new models on the same held-out split, inspect failure modes by category, build a schema-specific question-answer demonstration bank for production retrieval or adaptation, and export redacted review packets for governance teams.

The deployment path is designed for arbitrary enterprise schemas: teams provide read-only graph credentials, introspect labels/relationships/properties, choose a bounded categorical-value sampling policy, connect a local model endpoint, run a dry pass, scale generation, calibrate the judge, and export either raw internal artifacts or redacted review artifacts. This makes the FinBench/SNB/ICIJ study a reproducible evaluation setting rather than a hard-coded graph assumption.

8 Conclusion

PIPE-Cypher reframes Text2Cypher benchmarking as a private, repeatable enterprise workflow. The main lesson is that benchmark generation improves when Cypher constraints become executable gates: reverse grounding makes questions answerable, deterministic governance catches unsafe or schema-invalid queries, execution exposes empty or brittle candidates, diversity diagnostics reveal concentration, and a calibrated local judge provides a conservative semantic filter. This produces a benchmark that is not only larger, but more useful: it is balanced, auditable, refreshable, and able to reveal downstream model failures that syntax-only evaluation would hide.

Limitations

Execution validity does not guarantee semantic correctness. The completed 80-row, single-audit-sheet calibration suggests a conservative judge with no observed false accepts in the labeled sample, but the confidence interval is necessarily wider than the point estimate and larger multi-annotator audits may reveal additional failure modes. FinBench, SNB, and ICIJ are public enterprise-style proxies rather than a proprietary tenant graph, so the study tests the onboarding pattern but not every deployment constraint of a real organization. The full export is balanced by graph, category, and difficulty, yet query-signature diagnostics show residual template concentration from the seeded, execution-grounded generation strategy. The downstream few-shot result should be read as graph-specific example-bank conditioning: ordered and random same-category demonstrations often share query signatures, while the stricter no-signature control is lower and has a wide model-level interval despite improving the aggregate. Tenant-specific fine-tuning remains an engineering path enabled by the artifact, not a completed deployment claim in this paper. Generated artifacts may contain private schema details or sampled values, so organizations should apply privacy redaction and normal data governance controls before broad sharing.

Ethics Statement

PIPE-Cypher is designed for private benchmark generation under local-model deployment constraints. Organizations using it should restrict benchmark artifacts to authorized users, review

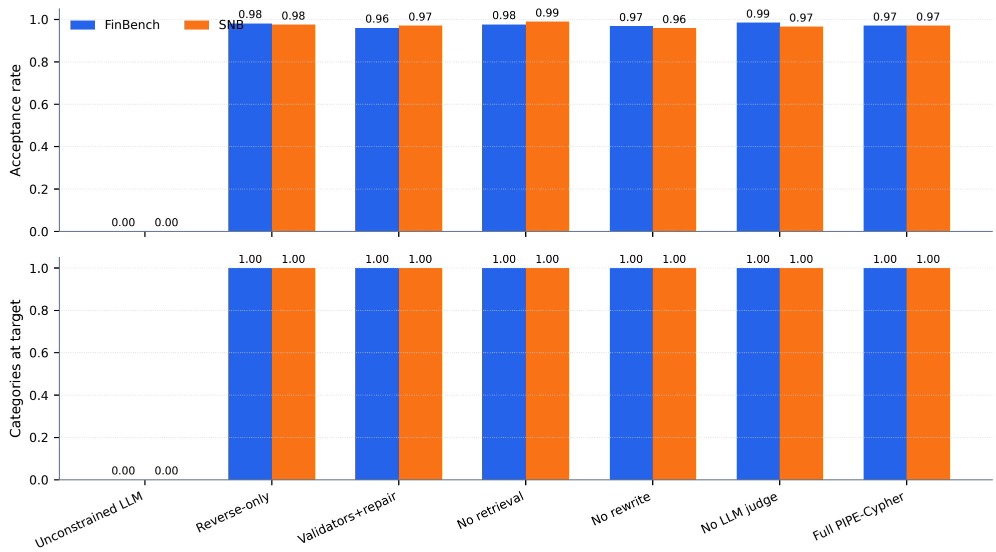

Figure 2: Target-100 ablation yield on FinBench and SNB. Unconstrained local generation is reported as a stress baseline with explicit attempt accounting; the execution-grounded governed variants reach all eight workload-category targets on both graphs. The takeaway is reliability of the grounded governance core, not that every optional module is needed for yield.

sampled values for sensitive content, and document whether judge calibration relied on human annotators.

References

Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, Ann Arbor, Michigan. Association for Computational Linguistics.

Vashu Chauhan, Shobhit Raj, Shashank Mujumdar, Avirup Saha, and Anannay Jain. 2025. Mind the query: A benchmark dataset towards Text2Cypher task. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1890–1905, Suzhou, China. Association for Computational Linguistics.

Matthew A. Jaro. 1989. Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida. *Journal of the American Statistical Association*, 84(406):414–420.

Moussa Kamal Eddine, Guokan Shang, Antoine Tixier, and Michalis Vazirgiannis. 2022. FrugalScore: Learning cheaper, lighter and faster evaluation metrics for automatic text generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1305–1318, Dublin, Ireland. Association for Computational Linguistics.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *Preprint*, arXiv:2411.07763.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119, San Diego, California. Association for Computational Linguistics.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.

Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. 2025. Text2Cypher: Bridging natural language and graph databases. In *Proceedings of the Workshop on Generative AI and*

Knowledge Graphs, pages 100–108, Abu Dhabi, UAE. International Committee on Computational Linguistics.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.

David P
üroja, Jack Waudby, Peter Boncz, and G
'abor Sz
'arnyas. 2023. The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations. *Preprint*, arXiv:2307.04820.

Shipeng Qi, Heng Lin, Zhihui Guo, G
'abor Sz
'arnyas, Bing Tong, Yan Zhou, Bin Yang, Jiansong Zhang, Zheng Wang, Youren Shen, Changyuan Wang, Parviz Peiravi, Henry Gabb, and Ben Steer. 2023. The LDBC financial benchmark. *Preprint*, arXiv:2306.15975.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. 2025. Auto-cypher: Improving LLMs on cypher generation via LLM-supervised generation-verification framework. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2: Short Papers*, pages 623–640, Albuquerque, New Mexico. Association for Computational Linguistics.

William E. Winkler. 1990. String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods*, pages 354–359. American Statistical Association.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*.

Xiuwen Zheng, Arun Kumar, and Amarnath Gupta. 2024. Generating cross-model analytics workloads using LLMs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4303–4307. ACM.

Zijie Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. 2025. SyntheT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 672–692, Abu Dhabi, UAE. Association for Computational Linguistics.

Jiongli Zhu, Haoquan Guan, Parjanya Prajakta Prashant, Nikki Lijing Kuang, Seyedeh Baharan Khatami, Canwen Xu, Xiaodong Yu, Yingyu Lin, Zhewei Yao, Yuxiong He, and Babak Salimi. 2026. Residual skill optimization for text-to-sql ensembles. *Preprint*, arXiv:2605.21792.

Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. 2018. Texygen: A benchmarking platform for text generation models. In *The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1097–1100. ACM.

A Additional Results

A.1 Downstream Stress-Test Interpretation

The downstream experiment is a benchmark-utility stress test. A useful enterprise benchmark should not merely reward syntactically valid Cypher; it should reveal when a model produces a query that parses, mentions plausible schema elements, and even executes, but answers the wrong operational question. The Qwen3.5-9B result has that shape: parse validity is 0.959 and schema validity is 0.905, while exact execution accuracy is only 0.189 on the full held-out split. This gap is evidence that PIPE-Cypher is measuring semantic graph-query competence rather than only query formatting.

The strategy-conditioned analysis makes the same point at operator level. Aggregation and single-hop queries account for most exact matches, whereas join-heavy, negation, ranking, and path-oriented examples expose wrong-direction, missing-context, execution-failure, and answer-mismatch errors. These zero-accuracy slices should not be read as a weakness of benchmark generation; they are the operational failure modes the benchmark is designed to surface. In an enterprise deployment, the same slices identify where retrieval examples, schema documentation, constrained decoding, or tenant-specific fine-tuning data are likely to matter.

We therefore ran the multi-model transfer stress test on the same fixed held-out split. The suite compares local general instruction, code-tuned, Cypher instruction, and Text2Cypher-finetuned models while keeping the schema prompt, evaluation backend, train/dev/test split, and execution metrics fixed.

The result is consistent on the transfer claim: zero-shot execution accuracy is weak across model families, including public Text2Cypher-tuned checkpoints. Demonstration-bank controls close much of the gap for compatible model families such as Qwen, Qwen-Coder, stable-cypher, and one Gemma Text2Cypher LoRA, but several fine-tuned checkpoints emit malformed or non-executable few-shot outputs. The model-level uncertainty table makes the claim deliberately conservative: ordered and random example-bank controls have positive checkpoint-level intervals, whereas the stricter no-signature control improves the mean but remains model-family dependent. This supports a practical industry reading of the benchmark: it is not only an evaluation artifact, but also a source of graph-specific examples for retrieval systems and future tenant-specific fine-tuning experiments, while still exposing model-family failures that require engineering attention.

The same result suggests a deployment pattern beyond static evaluation. An enterprise can index accepted PIPE-Cypher triples—user-style question, governed Cypher, schema slice, execution result sample, difficulty features, and validation metadata—as a private question-answer RAG bank. At generation time, a Text2Cypher system can retrieve examples matched by schema elements, relationship pattern, workload category, and historical user question phrasing. Table 8 shows why this matters: mean execution accuracy rises from 0.139 zero-shot to 0.245 under the scored no-signature control, and to 0.380/0.378 under ordered/random same-category example banks. Table 9 adds a checkpoint-level uncertainty check, while Table 10 makes the corresponding validity risk explicit: ordered and random conditions have substantial query-signature overlap, so they should be interpreted as example-bank conditioning rather than leakage-free retrieval generalization. This also makes PIPE-Cypher a useful substrate for residual-skill systems: recent Text-to-SQL work optimizes complementary agent skills on residual failures and reports large gains on Spider2-Lite across Snowflake and BigQuery (Zhu et al., 2026). PIPE-Cypher supplies the missing enterprise Cypher side of that loop: private, executable residual examples, failure taxonomies, and schema-grounded demonstrations that can drive retrieval policies, skill prompts, and future adaptation without exposing tenant data externally. We do not claim a completed tenant fine-tuning experiment

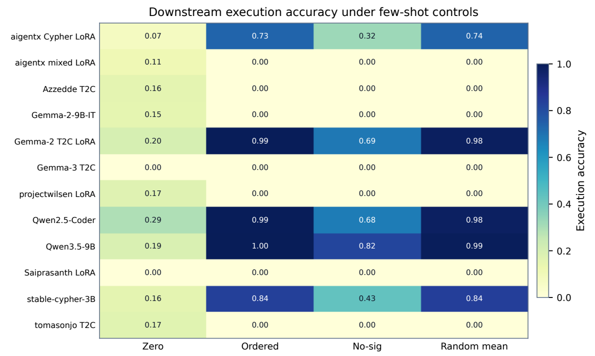

Figure 3: Execution accuracy for the 12 local downstream models under zero-shot and few-shot control modes. The heatmap highlights both findings: schema-specific examples can sharply help compatible models, and several public fine-tuned checkpoints still fail under enterprise-style Cypher prompting.

here; the claim is that PIPE-Cypher creates the audited residual examples such systems require.

Model	Tuning	Zero	Ordered	No-sig	Random μ	Random σ	Best
aigentx/Llama-3.1-8B Cypher LoRA	Cypher LoRA	0.074	0.733	0.321	0.735	0.011	random mean (0.735)
aigentx/Llama-3.1-8B Cypher mixed LoRA	Cypher mixed LoRA	0.111	0.000	0.000	0.000	0.000	no gain
Azzedde/llama3.1-8b-text2cypher	Text2Cypher fine-tuned	0.155	0.000	0.000	0.000	0.000	no gain
Gemma-2-9B-IT	general instruction	0.152	0.000	0.000	0.000	0.000	no gain
neo4j/Gemma-2-9B Text2Cypher LoRA	Text2Cypher LoRA	0.199	0.993	0.686	0.982	0.002	ordered (0.993)
neo4j/Gemma-3-4B Text2Cypher	Text2Cypher fine-tuned	0.000	0.000	0.000	0.000	0.000	no gain
projectwilsen/Llama-3.1-8B Text2Cypher LoRA	Text2Cypher LoRA	0.169	0.000	0.000	0.000	0.000	no gain
Qwen2.5-Coder-7B-Instruct	code instruction	0.291	0.993	0.676	0.980	0.000	ordered (0.993)
Qwen3.5-9B	general instruction	0.189	0.997	0.821	0.990	0.003	ordered (0.997)
Saiprasanth15/Llama-3.1-8B Text2Cypher LoRA	Text2Cypher LoRA	0.000	0.000	0.000	0.000	0.000	no gain
ragraph-ai/stable-cypher-instruct-3b	Cypher instruction	0.155	0.838	0.432	0.843	0.008	random mean (0.843)
tomasonjo/text2cypher-demo-16bit	Text2Cypher fine-tuned	0.166	0.000	0.000	0.000	0.000	no gain

Table 8: Few-shot demonstration-bank controls for local downstream Text2Cypher evaluation. Ordered uses the deterministic same-graph, same-category example bank; scored excludes exact query-signature matches and near-duplicate questions; random reports the mean and standard deviation across seeds 13, 17, and 23. “No gain” means no few-shot control exceeded that model’s zero-shot execution accuracy.

Control	Mean acc.	Δ vs zero	95% Δ CI	Models improved
Ordered same-category	0.380	0.241	[0.014, 0.489]	5/12
Scored no-signature	0.245	0.106	[-0.041, 0.266]	5/12
Random same-category mean	0.378	0.239	[0.013, 0.477]	5/12

Table 9: Model-level paired bootstrap uncertainty for downstream few-shot controls. The unit of resampling is the local checkpoint, not an individual question, so the interval is a conservative check on whether gains are broad across model families. Zero-shot mean execution accuracy is 0.139.

Selection mode	Rows	Demos	Sig. match	High sim.	Mean sim.
ordered	296	1480	0.888	0.393	0.846
scored no-sig	296	1480	0.000	0.000	0.593
random seed 13	296	1480	0.870	0.406	0.840
random seed 17	296	1480	0.872	0.399	0.836
random seed 23	296	1480	0.882	0.410	0.844

Table 10: Few-shot leakage controls for downstream demonstration-bank evaluation. The held-out split has 0 exact train/test question overlaps and 295 train/test query-signature overlaps; selection-mode rates show how often retrieved demonstrations share the test query signature or exceed the 0.90 normalized-question similarity threshold.

Graph	Nodes	Relationships	Labels	Rel. types	Paper status
FinBench	10,006	57,622	5	9	reported
SNB	34,735	70,842	14	15	reported
ICIJ Offshore Leaks	2,016,523	3,339,267	5	14	onboarding audit

Table 11: Study graph statistics. ICIJ Offshore Leaks is used as a public third-graph onboarding audit for arbitrary finance/compliance schemas beyond the two LDBC study workloads.

ICIJ onboarding property	Value
Graph nodes / relationships	2,016,523 / 3,339,267
Labels / relationship types	5 / 14
Generated / accepted	983 / 800
Acceptance rate	0.814
Categories at target	8/8
Paper audit	ready
Sparse schema-derived accepts	complex agg. 97, negation 28, ranking 98

Table 12: ICIJ Offshore Leaks third-graph onboarding audit. The public finance/compliance graph tests arbitrary-schema generation beyond the two LDBC study workloads; raw values remain outside the paper artifacts.

Metric	Value
Question Distinct-1	0.041
Question Distinct-2	0.088
Question self-BLEU-2 (sampled)	0.826
Unique query-signature ratio	0.010
Top query-signature share	0.083
Category normalized entropy	1.000
Graph-category normalized entropy	0.980
Difficulty normalized entropy	1.000
Label coverage	0.412
Relationship-type coverage	0.375
Property-name coverage	0.407
Unique grounded-value ratio	0.373
Grounded values exactly quoted	0.826
Aggregation / negation / ordering rates	0.500 / 0.125 / 0.125

Table 13: Diversity diagnostics for the full exported benchmark. Distinct-n follows text-generation usage; self-BLEU is lower when questions are less redundant; grounded-value metrics are aggregate-only and do not list raw values.

Signature ID	Count	Share	Canonical preview
f15b226863	250	0.083	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account)-[t:transfer_to]->(:account) return distinct sum(<var>.amount) as to
06f23c9775	250	0.083	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account) return distinct <var>.accountid as accountid, <var>.accounttype as
15552613ff	250	0.083	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account) return distinct count(distinct a) as accountcount
375606ce8c	250	0.083	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account)-[:transfer_to]->(<var>:account) return distinct <var>.accountid as
4a1918b4a5	248	0.083	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account)-[t:transfer_to]->(:account) with src, sum(<var>.amount) as totalamo
f2374ff7d1	226	0.075	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account) where not (a)-[:transfer_to]->(:account) return distinct <var>.acco
aa411b54b1	158	0.053	match (<var>:account {accountid: <str>})-[:transfer_to]->(:account) return distinct count(distinct src) > <num> as hasoutgoingtransfer
f384aa977a	137	0.046	match (<var>:person {personname: <str>})-[:own_account]->(<var>:account)-[:transfer_to*<num>..<num>]->(<var>:account) return distinct <var>.

Table 14: Top canonical query signatures in the full export. Literals, numbers, and variables are normalized before counting, so this table measures template concentration rather than raw value reuse.

Primary strategy	Examples	Share	Rel. patterns	Return arity	Downstream exec. acc.
Aggregation	1125	0.375	1.42	1.00	0.396
Join-heavy	375	0.125	2.33	2.33	0.000
Negation	375	0.125	2.08	3.00	0.000
Order/rank	375	0.125	1.99	3.34	0.000
Path	375	0.125	1.37	3.00	0.000
Single hop	375	0.125	1.00	2.33	0.324

Table 15: Cypher strategy diagnostics over the full 3,000-example export. Strategy tags are derived from generated Cypher structure rather than from category labels; downstream execution accuracy is reported on the full held-out test split when a strategy appears there.

Failure bucket	Count	Share of rejected
Diversity/duplicate control	1,335	0.751
Empty result	374	0.210
Judge semantic reject	66	0.037
Schema invalid	2	0.001

Table 16: Failure taxonomy over full-run generation candidates before benchmark export. Accepted examples are excluded from the bucket shares.

PIPE-Cypher category	Closest MTQ category	Added enterprise distinction
Simple retrieval	SR	Direct node/edge lookup with exact filters.
Complex retrieval	CR	Multi-hop or multi-pattern retrieval.
Simple aggregation	SA	Single aggregation such as count/min/max/average.
Complex aggregation	CA	Grouped or multi-stage aggregation over graph neighborhoods.
Boolean existence	EQ	Precise yes/no or existence answer.
Negation/difference	CR	Absence, anti-join, or difference query.
Path/temporal transaction	CR/CA	Temporal or path-oriented transaction neighborhood.
Ranking/top-k	SA/CA	Ordered top-k query with explicit limit.

Table 17: Category crosswalk to Mind the Query. PIPE-Cypher keeps the familiar retrieval/aggregation/evaluation-query structure while adding enterprise workloads such as negation, temporal paths, and ranking.

Gate or ledger	Count	Denominator
Export accepted	3,000	3,000
Read-only safety	3,000	3,000
Syntax validity	3,000	3,000
Schema/value validity	3,000	3,000
Execution success	3,000	3,000
LLM judge pass	3,000	3,000
Rejected candidates logged	1,777	4,777

Table 18: PIPE-Cypher validation cascade for the full export and its logged candidate ledger. Unlike Mind the Query, human review is calibration-only.

Profile	Added constraint	Intended evidence
Schema-only	Use only visible schema.	Baseline for schema-grounded prompting.
Instructions	Exact values, datatype rules, no nested aggregations.	Targets MTQ-style prompt refinements.
Examples	Placeholderized retrieved NL-Cypher pairs.	Tests whether examples help without extra governance.
Examples + instructions	Few-shot plus explicit rules.	Closest controlled analogue to MTQ Table 5.
Full governed	Production-derived Cypher hints, AST-safe rewrites, judge gate.	PIPE-Cypher production setting.

Table 19: Prompt profiles implemented for Mind-the-Query-style prompt-factorial evaluation. Results are reported only for completed, audited target-50-or-larger suites.

Dimension	Mind the Query	PIPE-Cypher
Generation review gate	Manual logical review	Deterministic gates + local LLM judge
Human effort	Reported 1,400 person-hours	80-row post-hoc judge calibration audit
Private values	Public benchmark values	Configurable sampling and redacted export
Refresh	Static dataset release	Rerunnable private benchmark factory
Model endpoint	Gemini in their reported pipeline	Local Qwen3.5-9B endpoint

Table 20: Industry deployment contrast with Mind the Query. PIPE-Cypher focuses on private refreshable benchmark generation rather than a one-time public dataset.

Setting	Graph	Attempts	Records	Accepted	Acceptance	Categories at target
Unconstrained LLM	FinBench	422	422	200	0.474	2/8
Unconstrained LLM	SNB	2,000	2,000	50	0.025	0/8
Reverse-only	FinBench	815	815	800	0.982	8/8
Reverse-only	SNB	820	820	800	0.976	8/8
Validators+repair	FinBench	833	833	800	0.960	8/8
Validators+repair	SNB	824	824	800	0.971	8/8
No retrieval	FinBench	819	819	800	0.977	8/8
No retrieval	SNB	809	809	800	0.989	8/8
No rewrite	FinBench	826	826	800	0.969	8/8
No rewrite	SNB	834	834	800	0.959	8/8
No LLM judge	FinBench	812	812	800	0.985	8/8
No LLM judge	SNB	828	828	800	0.966	8/8
Full PIPE-Cypher	FinBench	824	824	800	0.971	8/8
Full PIPE-Cypher	SNB	824	824	800	0.971	8/8

Table 21: Live target-100 ablation evidence with local Qwen3.5-9B. Governed graph runs target 100 accepted examples per category; the unconstrained row is a stress baseline reported with explicit attempt accounting.

Setting	Graph	Read-only	Syntax	Schema	Exec.	Judge/post-hoc
Unconstrained LLM	FinBench	1.000	1.000	0.806	0.723	0.557
Unconstrained LLM	SNB	1.000	0.998	0.599	0.478	0.144
Reverse-only	FinBench	1.000	1.000	0.982	0.982	0.982
Reverse-only	SNB	1.000	1.000	0.998	0.998	0.976
Validators+repair	FinBench	1.000	1.000	1.000	1.000	0.960
Validators+repair	SNB	1.000	1.000	0.998	0.998	0.971
No retrieval	FinBench	1.000	1.000	1.000	1.000	0.977
No retrieval	SNB	1.000	1.000	0.998	0.998	0.989
No rewrite	FinBench	1.000	1.000	1.000	1.000	0.969
No rewrite	SNB	1.000	1.000	0.998	0.998	0.959
No LLM judge	FinBench	1.000	1.000	1.000	1.000	0.985
No LLM judge	SNB	1.000	1.000	0.998	0.998	0.966
Full PIPE-Cypher	FinBench	1.000	1.000	1.000	1.000	0.971
Full PIPE-Cypher	SNB	1.000	1.000	0.998	0.998	0.971

Table 22: Quality-gate rates for the live target-100 ablation suite. Rates are computed over all generated records in each graph/setting; for no-judge settings, the judge column is a post-hoc scoring diagnostic.

Setting	Graph	Suites	Target cov.	Acceptance	Cat. target	Exec.	Judge
No LLM judge	FinBench	3/3	1.000	0.994±0.008	1.000	1.000	0.994
No retrieval	FinBench	3/3	1.000	0.967±0.031	1.000	1.000	0.967
No rewrite	FinBench	3/3	1.000	0.969±0.023	1.000	1.000	0.969
Full PIPE-Cypher	FinBench	3/3	1.000	0.975±0.016	1.000	1.000	0.975
Reverse-only	FinBench	3/3	1.000	0.994±0.011	1.000	0.994	0.994
Unconstrained LLM	FinBench	3/3	0.000	0.000	0.000	0.000	0.000
Validators+repair	FinBench	3/3	1.000	0.987±0.023	1.000	1.000	0.987
No LLM judge	SNB	3/3	1.000	0.982±0.015	1.000	0.996	0.982
No retrieval	SNB	3/3	1.000	0.993±0.004	1.000	0.996	0.993
No rewrite	SNB	3/3	1.000	0.983±0.021	1.000	0.996	0.983
Full PIPE-Cypher	SNB	3/3	1.000	0.987±0.014	1.000	0.996	0.987
Reverse-only	SNB	3/3	1.000	0.989±0.011	1.000	0.996	0.989
Unconstrained LLM	SNB	3/3	0.000	0.000	0.000	0.000	0.000
Validators+repair	SNB	3/3	1.000	0.987±0.014	1.000	0.996	0.987

Table 23: Target-size and repeated-seed ablation sensitivity. Target coverage normalizes accepted examples by each suite's planned graph/category target, so target-50 and target-100 suites can be compared without treating larger raw counts as quality gains.

Audit packet property	Value
Rows	80
Judge accept / reject	40 / 40
FinBench / SNB rows	48 / 32
Easy / medium rows	26 / 54
Labeled rows	80
Calibration status	complete
Agreement / κ	0.800 / 0.600
Judge precision (95% CI)	1.000 (0.912–1.000)
Judge recall (95% CI)	0.714 (0.585–0.816)
False-accept rate (95% CI)	0.000 (0.000–0.138)
False-reject rate (95% CI)	0.286 (0.184–0.415)

Table 24: Post-hoc judge calibration packet coverage and agreement. Human labels calibrate the automated gate after generation and are not used as a generation gate.

Metric	Point	95% CI	SE	N
Parse valid	0.959	[0.936, 0.980]	0.012	296
Schema valid	0.905	[0.872, 0.939]	0.017	296
Execution success	0.622	[0.564, 0.676]	0.028	296
Execution accuracy	0.189	[0.145, 0.233]	0.022	296
Answer F1	0.189	[0.149, 0.236]	0.023	296

Table 25: Bootstrap uncertainty for downstream Text2Cypher evaluation on the full exported test split. Intervals use row-level resampling with a fixed seed and are intended for appendix reporting.

Downstream outcome	Count	Share of incorrect
Answer mismatch	128	0.533
Execution failed	72	0.300
Schema invalid	28	0.117
Parse invalid	12	0.050

Table 26: Downstream Text2Cypher failure taxonomy for local Qwen3.5-9B on the full exported test split. Shares exclude exact-answer matches.

Metric	PIPE-Cypher support	Primary citation
ROUGE-1/2/L	Reference overlap over serialized answer sets and query strings	(Lin, 2004)
BLEU	Sentence-level reference overlap with brevity penalty	(Papineni et al., 2002)
METEOR	Exact-token METEOR-style precision/recall with fragmentation penalty	(Banerjee and Lavie, 2005)
BERTScore	Optional contextual-embedding similarity when installed	(Zhang et al., 2020)
FrugalScore	Optional efficient learned NLG metric when installed	(Kamal Eddine et al., 2022)
Cosine	Token-count vector cosine similarity	(Manning et al., 2008)
Jaro-Winkler	Character-level string similarity for near-match diagnostics	(Jaro, 1989; Winkler, 1990)
Exact match	Raw and normalized string exact match	(Rajpurkar et al., 2016)

Table 27: Supplementary reference-based text metrics supported by the PIPE-Cypher evaluator. These metrics are useful for debugging answer rendering, paraphrase sensitivity, and near-match behavior, but they do not replace execution accuracy or answer-set F1 for Text2Cypher correctness. BERTScore and FrugalScore are optional integrations because they require additional metric/model packages.

B Claim–Evidence Map

This section maps the main paper claims to the strongest current evidence and to the remaining risks. This is intended as a reviewer-facing audit surface: claims with pending evidence remain marked as such rather than being folded into the main results.

1. **Claim 1.** PIPE-Cypher can generate a private, executable NL-to-Cypher benchmark over enterprise-style property graphs using local models.

   Evidence. The full local Qwen3.5-9B run exported 3,000 accepted examples: 2,000 FinBench and 1,000 SNB, with 375 examples in every planned workload category and all accepted examples passing read-only, syntax, schema, execution, non-empty-result, and judge gates.

   Key artifacts. benchmark export; manifest snapshot; paper table: benchmark export; paper table: full results

   Status. Supported by live local-model evidence.

   Risk. Generation quality may change with private enterprise schemas beyond FinBench/SNB.

2. **Claim 2.** Cypher-specific governance makes generated benchmark candidates safer and more auditable than prompt-only generation.

   Evidence. The pipeline validates read-only safety, schema-visible labels, node and relationship properties, relationships, observed relationship direction, categorical values, contextual return columns, parser-style structure, execution success, and LLM-judge review. In the full run, only 2 of 4,777 candidates were schema-invalid after deterministic governance, and all 3,000 exported examples passed the deterministic and judge gates.

   Key artifacts. code: validator.py; code: cypher_parser.py; code: question_constraints.py; snapshot: failure taxonomy; +1 more

   Status. Supported by code, tests, and full-run failure taxonomy.

   Risk. Semantic correctness still depends on execution evidence and judge review; the completed 80-row audit is a calibration sample rather than an exhaustive proof.

3. **Claim 3.** Reverse grounding and structured workload seeds are designed to improve reliability under local-model constraints.

   Evidence. Three FinBench/SNB ablation suites are complete and evidence-ready: the original target-50 suite, the corrected target-100 suite, and a seed-17 target-50 repeat. Each suite completed 14/14 graph/variant cells, reached every category target for all non-empty/non-unconstrained runs, and passed paper-readiness audits with logs, model IDs, code revisions, run summaries, collection manifests, and gate-rate availability. The target-100 suite is the detailed appendix ablation table/figure, while the three-suite comparison reports target-normalized coverage, acceptance variation, execution rates, and judge rates for stability.

   Key artifacts. script: run live ablation suite; script: monitor remote ablation suite; script: monitor remote ablation queue; script: collect remote ablation suite; +20 more

   Status. Supported by audited target-100 evidence and three-suite target-size/repeated-seed comparison.

   Risk. The repaired unconstrained local generation baseline produced accepted examples only in a narrow subset of categories: 200/422 accepted on FinBench across 2/8 categories and 50/2,000 accepted on SNB across 0/8 category targets. Comparisons against it should be framed as a gate-stress baseline rather than a tuned generation system.

4. **Claim 4.** The benchmark controls category and difficulty balance while exposing residual diversity limits.

   Evidence. The full export has perfect category balance, planned 2:1 FinBench/SNB graph mix, and a near-even easy/medium split. Diversity diagnostics report Distinct-n, sampled self-BLEU-2, query-signature diversity, normalized entropy, schema coverage, structural feature rates, and aggregate value-grounding diagnostics. The full export uses 1,115 unique grounded entity values and exactly quotes grounded values in 82.6% of examples with entity bindings, making template and value concentration visible instead of hidden.

Key artifacts. snapshot: diversity report; paper table: diversity; paper figure: diversity diagnostics; paper figure: full export distribution

Status. Supported by full-export statistics and diversity diagnostics.

Risk. Low query-signature diversity in the reported local-model run remains a limitation and ablation target.

5. **Claim 5.** The generated benchmark is useful for downstream Text2Cypher evaluation.

   Evidence. A 12-model local downstream evaluation over the 296-example full test split reports parse validity, schema validity, execution success, execution accuracy, answer F1, per-category breakdowns, bootstrap uncertainty for the primary Qwen3.5-9B baseline, and row-level error taxonomies. Zero-shot execution accuracy ranges from 0.000 to 0.291 with a mean of 0.139. Few-shot control runs over the same split improve the mean to 0.245 for a scored no-signature control that excludes exact query-signature matches and near-duplicate questions, and to 0.380/0.378 for ordered/random same-category example banks. A model-level paired bootstrap reports wide uncertainty for the stricter no-signature control and positive intervals for ordered/random example-bank conditioning. The leakage audit reports the corresponding selected-demonstration signature and similarity rates, so example-bank gains are not treated as leakage-free retrieval generalization.

   Key artifacts. downstream control summary; few-shot control uncertainty; few-shot leakage control audit; downstream uncertainty; downstream error report; +8 more

   Status. Supported by live downstream evaluation against FinBench/SNB databases with complete 12-model zero-shot/control summaries.

   Risk. Several local fine-tuned checkpoints remain brittle under few-shot prompting; the current study shows example-bank utility and transfer failure modes, not a completed tenant-specific fine-tuning result. The stricter no-signature improvement is an aggregate signal with broad model-family uncertainty rather than a universal model-level gain.

6. **Claim 6.** Automated LLM-judge review can serve as a generation gate with post-hoc human calibration while still exposing judge risk.

   Evidence. The pipeline uses deterministic validation plus LLM-judge review as the generation gate. A post-hoc 80-row judge audit packet preserves 40 judge accepts, 40 judge rejects, both graphs, and all eight categories. The completed human audit reports 80.0% agreement, Cohen's kappa 0.60, balanced accuracy 0.857, judge precision 1.00, judge specificity 1.00, judge recall 0.714, zero false accepts, and 16 false rejects. The analyzer requires complete labels for paper-facing calibration, and the tracked summary avoids committing raw value-bearing audit rows.

   Key artifacts. code: judge.py; code: calibration.py; code: judge_audit_packet.py; script: create judge annotation sheets; +6 more

   Status. Supported by completed human audit summary.

   Risk. The judge appears conservative on this sample; larger audits or different enterprise schemas may reveal false accepts.

7. **Claim 7.** PIPE-Cypher supports arbitrary enterprise schema onboarding with configurable privacy redaction and value-sampling policies.

   Evidence. The repo includes an enterprise deployment guide, an enterprise template config, privacy config fields, schema introspection that reads categorical-value thresholds, maximum sampled value length, and omitted-property patterns from config, deterministic redaction helpers, and a redacted benchmark export CLI. The redaction policy replaces quoted Cypher literals, question mentions, entity values, and string-valued result samples with stable placeholders while preserving query structure and gate metadata. The repo now also includes an ICIJ Offshore Leaks onboarding profile, config, download/load helpers, graph plan, schema-derived sparse-category templates, and a completed sanitized target-100 onboarding audit on a public finance/compliance graph beyond the LDBC study workloads.

Key artifacts. research note: enterprise deployment guide; research note: icij offshore-leaks graph plan; enterprise_template.yaml; schema_icij_offshoreleaks.json; +21 more

Status. Supported by docs, config, code, deterministic tests, a concrete third-graph onboarding plan, and a completed sanitized ICIJ target-100 live onboarding run with 800/983 accepted examples and 8/8 categories at target.

Risk. ICIJ is public rather than private/anonymized enterprise data. The strongest industry validation would still come from a real tenant graph under the same audit protocol.

C Prompt Contracts

PIPE-Cypher treats prompts as versioned implementation artifacts. The list summarizes the prompt contracts used for generation, repair, judging, and downstream evaluation; hashes fingerprint the full prompt constants in the codebase.

1. **Template generation.** Stage: Workload proposal. SHA-256: `10b25b`.

   Contract. Schema-only labels, relationships, properties, and categorical values; Realistic enterprise analyst wording; At most two typed slots and JSON-only output

2. **Reverse binding.** Stage: Graph grounding. SHA-256: `48e949`.

   Contract. Read-only MATCH/WHERE/RETURN DISTINCT/LIMIT only; Slot variables named exactly as requested; Forward relationship directions from the schema

3. **Cypher generation.** Stage: Candidate query. SHA-256: `61c557`.

   Contract. Only schema-visible constructs and observed directions; RETURN DISTINCT for set returns and exact equality for quoted values; Context columns, categorical hints, placeholderized retrieval, and no writes

4. **Repair.** Stage: Validation feedback. SHA-256: `56c419`.

   Contract. Preserve question intent while fixing validation or execution issues; Keep query read-only and schema-grounded; Return only corrected Cypher

5. **LLM judge.** Stage: Quality gate. SHA-256: `421c7b`.

   Contract. Inputs include question, Cypher, schema slice, execution rows, and validation summary; Strict JSON scores for ambiguity, semantic alignment, schema use, and difficulty; Categorical values constrain query literals, not observed result-row values; Pass only useful, unambiguous enterprise benchmark examples

6. **Downstream Text2Cypher.** Stage: Model evaluation. SHA-256: `4c07ff`.

   Contract. Read-only Cypher only; Schema-visible constructs and exact direction preservation; RETURN DISTINCT, count/ranking rules, and no explanations

D Representative Accepted Examples

The examples below are selected in stable identifier order from the tracked full-export snapshot, one per graph/category cell when available. They show the natural-language question, accepted Cypher, structural tags, gate status, and a bounded execution-result sample.

1. **FinBench / Boolean Existence / easy.** *Question:* Does account '187743809466009406' have any outgoing transfer?

   ```
   MATCH (src:Account {accountId:
   '187743809466009406'})
   -[:TRANSFER_TO]->
   (:Account)
   RETURN DISTINCT COUNT(DISTINCT src)
   > 0
   AS HasOutgoingTransfer
   ```

   Structure: single_hop, aggregation.

   Relationships: TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {HasOutgoingTransfer: True}; observed rows: 1

2. **FinBench / Complex Aggregation / medium.** *Question:* What is the total transferred amount from accounts owned by person 'Gwar'?

   ```
   MATCH (p:Person {personName:
   'Gwar'})
   -[:OWN_ACCOUNT]->
   (src:Account)-[t:TRANSFER_TO]->
   (:Account)
   RETURN DISTINCT SUM(t.amount) AS
   TotalTransferredAmount
   ```

   Structure: join_heavy, aggregation.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {TotalTransferredAmount: 14972318.41}; observed rows: 1

3. **FinBench / Complex Retrieval / easy.** *Question:* Which accounts received transfers from accounts owned by person 'Zof'?

   ```
   MATCH (p:Person {personName:
   'Zof'})
   -[:OWN_ACCOUNT]-> (src:Account)
   -[:TRANSFER_TO]-> (dst:Account)
   RETURN DISTINCT dst.accountId AS
   AccountId, dst.accountType AS
   AccountType, dst.isBlocked AS
   IsBlocked
   LIMIT 300
   ```

Structure: join_heavy, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 3

4. **FinBench / Negation Difference / medium.** *Question:* Which accounts owned by person 'Kant' have not sent any transfers?

```
MATCH (p:Person {personName:
'Kant'})
-[:OWN_ACCOUNT]-> (a:Account)
WHERE NOT (a) -[:TRANSFER_TO]->
(:Account)
RETURN DISTINCT a.accountId AS
AccountId, a.accountType AS
AccountType,
a.isBlocked AS IsBlocked
LIMIT 300
```

Structure: join_heavy, negation, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4739757132830738341, AccountType: merchant account, IsBlocked: False}; observed rows: 1

5. **FinBench / Path Temporal / medium.** *Question:* Which accounts can receive money within two transfer hops from accounts owned by person 'Sossamon'?

```
MATCH (p:Person {personName:
'Sossamon'}) -[:OWN_ACCOUNT]->
(src:Account) -[:TRANSFER_TO*1..2]->
(dst:Account)
RETURN DISTINCT dst.accountId AS
AccountId, dst.accountType AS
AccountType, dst.isBlocked AS
IsBlocked
LIMIT 300
```

Structure: join_heavy, path, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 4

6. **FinBench / Ranking Topk / medium.** *Question:* For accounts owned by person 'Barry', which account sent the highest total transfer amount?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]->
(src:Account)-[t:TRANSFER_TO]->
(:Account)
WITH src, SUM(t.amount) AS
totalAmount
RETURN DISTINCT src.accountId AS
AccountId, src.accountType AS
AccountType, src.isBlocked AS
IsBlocked,
totalAmount
ORDER BY totalAmount DESC
LIMIT 1
```

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

7. **FinBench / Simple Aggregation / easy.** *Question:* How many accounts are owned by person 'Kaewsuktae'?

```
MATCH (p:Person {personName:
'Kaewsuktae'}) -[:OWN_ACCOUNT]->
(a:Account)
RETURN DISTINCT COUNT(DISTINCT a) AS
AccountCount
```

Structure: single_hop, aggregation.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountCount: 1}; observed rows: 1

8. **FinBench / Simple Retrieval / easy.** *Question:* Which accounts are owned by person 'Barry'?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]-> (a:Account)
RETURN DISTINCT a.accountId AS
AccountId, a.accountType AS
AccountType,
a.isBlocked AS IsBlocked
LIMIT 300
```

Structure: single_hop, bounded_result.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

9. **SNB / Boolean Existence / medium.** *Question:* Does person with id 6597069766828 like any post?

```
MATCH (p:Person {id:
6597069766828})
OPTIONAL MATCH (p) -[:LIKES]->
(post:Post)
RETURN DISTINCT COUNT(post) > 0 AS
LikesAnyPost
```

Structure: single_hop, aggregation, optional.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {LikesAnyPost: True}; observed rows: 1

10. **SNB / Complex Aggregation / medium.** *Question:* How many distinct posts are in forums joined by person with id 6597069766845?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person {id: 6597069766845})
MATCH (forum) -[:CONTAINER_OF]->
(post:Post)
RETURN DISTINCT COUNT(DISTINCT post)
AS
JoinedForumPostCount
```

Structure: join_heavy, aggregation.

Relationships: CONTAINER_OF, HAS_MEMBER.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {JoinedForumPostCount: 1}; observed rows: 1

11. **SNB / Complex Retrieval / easy.** *Question:* Which people are members of forums containing posts tagged 'Manuel_Noriega'?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person), (forum)
-[:CONTAINER_OF]->
(post:Post) -[:HAS_TAG]-> (tag:Tag
{name: 'Manuel_Noriega'})
RETURN DISTINCT p.id AS PersonId
LIMIT 200
```

Structure: join_heavy, bounded_result.

Relationships: CONTAINER_OF, HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PersonId: 4398046511220}; observed rows: 19

12. **SNB / Negation Difference / medium.** *Question:* Which members of forum 'Wall of Celso Oliveira' have not liked any post?

```
MATCH (forum:Forum {title: 'Wall of
Celso Oliveira'}) -[:HAS_MEMBER]->
(p:Person)
WHERE NOT (p) -[:LIKES]-> (:Post)
RETURN DISTINCT p.id AS PersonId,
p.firstName AS FirstName, p.lastName
AS
LastName
LIMIT 200
```

Structure: join_heavy, negation, bounded_result.

Relationships: HAS_MEMBER, LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {FirstName: K., LastName: Sen, PersonId: 94}; observed rows: 1

13. **SNB / Path Temporal / easy.** *Question:* Which people are within two knows hops of person with id 4398046511136?

```
MATCH (src:Person {id:
4398046511136})
-[:KNOWS*1..2]-> (dst:Person)
RETURN DISTINCT dst.id AS PersonId,
dst.firstName AS FirstName,
dst.lastName
AS LastName
LIMIT 200
```

Structure: single_hop, path, bounded_result.

Relationships: KNOWS.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {FirstName: Rafael, LastName: Fernández, PersonId: 4398046511333}; observed rows: 25

14. **SNB / Ranking Topk / medium.** *Question:* Among forums tagged 'James_K._Polk', which forum has the most members?

```
MATCH (forum:Forum) -[:HAS_TAG]->
(tag:Tag {name: 'James_K._Polk'})
MATCH (forum) -[:HAS_MEMBER]->
(person:Person)
WITH forum, COUNT(DISTINCT person)
AS
memberCount
RETURN DISTINCT forum.title AS
ForumTitle, memberCount
ORDER BY memberCount DESC
LIMIT 1
```

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {ForumTitle: Wall of Javed Khan, memberCount: 33}; observed rows: 1

15. **SNB / Simple Aggregation / easy.** *Question:* How many posts are tagged 'Vietnam'?

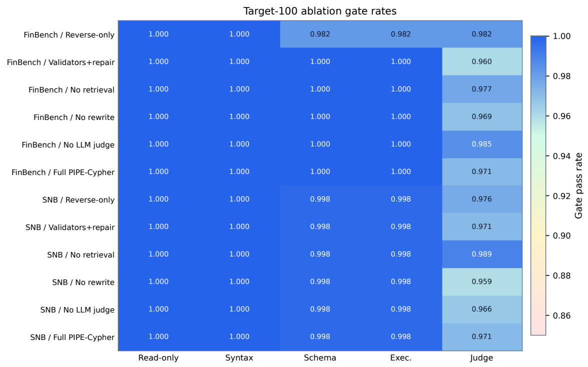

Figure 4: Gate-rate heatmap for the audited target-100 ablation suite. Deterministic read-only and syntax gates are saturated; execution and judge rates expose the remaining quality differences across graph and pipeline variants.

```
MATCH (post:Post) -[:HAS_TAG]->
(tag:Tag
{name:  'Vietnam'})
RETURN DISTINCT COUNT(DISTINCT post)
AS
PostCount
```

Structure: single_hop, aggregation.

Relationships: HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostCount: 1}; observed rows: 1

16. **SNB / Simple Retrieval / easy.** *Question:* Which post IDs did person with id 4398046511124 like?

```
MATCH (p:Person {id:
4398046511124})
-[:LIKES]-> (post:Post)
RETURN DISTINCT post.id AS PostId
LIMIT 200
```

Structure: single_hop, bounded_result.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostId: 343597385744}; observed rows: 5

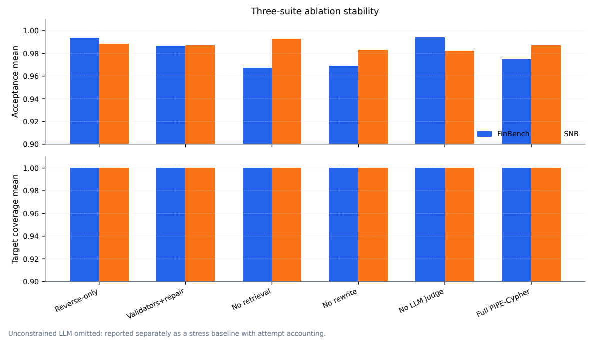

Figure 5: Three-suite ablation stability over the original target-50 suite, corrected target-100 suite, and seed-17 target-50 repeat. Target-normalized coverage stays at 1.000 for all non-unconstrained cells.

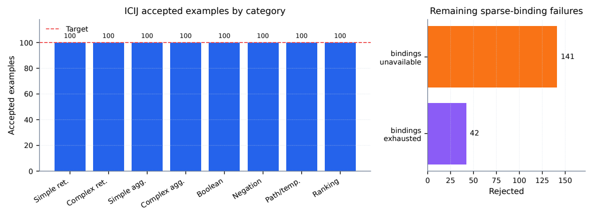

Figure 6: ICIJ Offshore Leaks onboarding audit. Schema-derived sparse-category templates recover balanced target-100 coverage on a public finance/compliance graph beyond the two LDBC workloads.

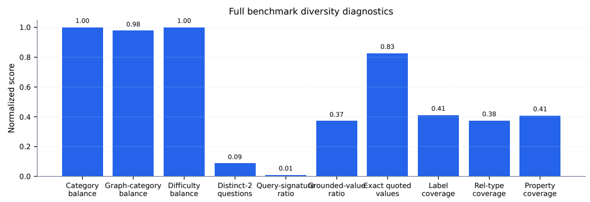

Figure 7: Diversity diagnostics for the full 3,000-example export. Category, graph-category, and difficulty balance are strong by construction; schema coverage and query-signature diversity expose remaining concentration from the seeded-template run.

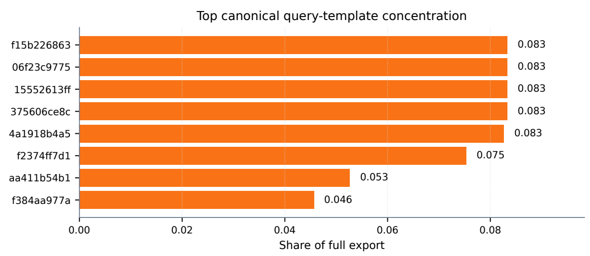

Figure 8: Top canonical query-template concentration in the full export. The plot makes template reuse auditable rather than hidden behind aggregate category balance.

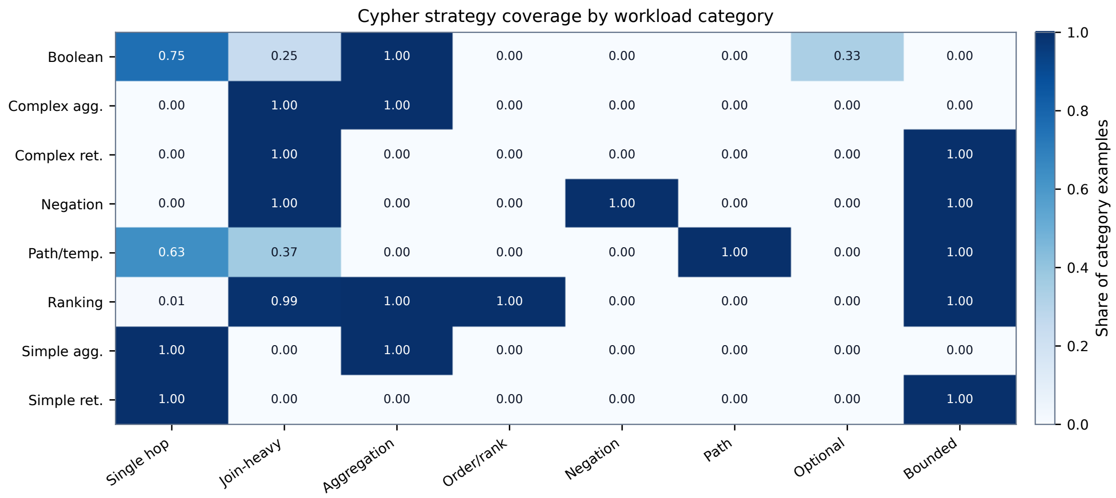

Figure 9: Cypher strategy coverage by workload category. Category balancing does not by itself guarantee operator coverage; the strategy matrix shows where the benchmark exercises single-hop retrieval, joins, aggregation, ordering, negation, path patterns, optional matches, and bounded-result queries.

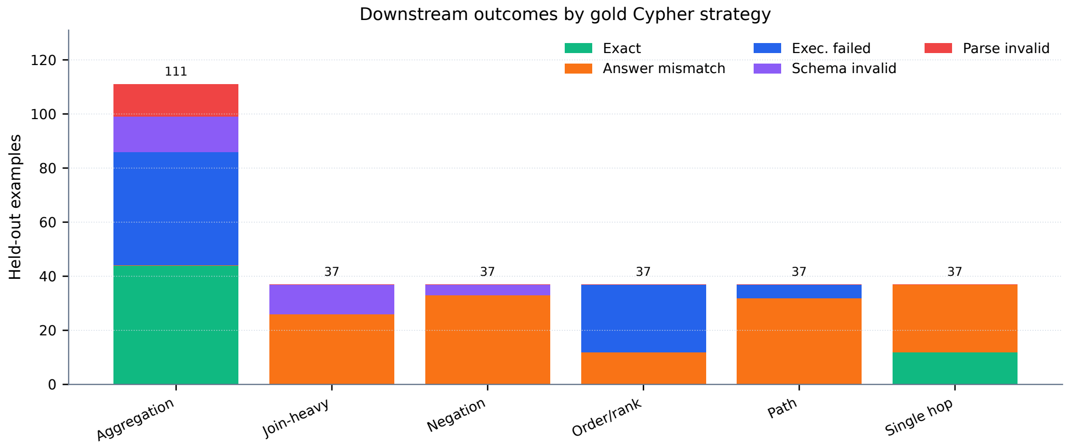

Figure 10: Downstream outcomes by gold Cypher strategy on the full held-out test split. The local Qwen3.5-9B baseline fails differently across strategies: aggregation mixes exact matches with execution failures, while join-heavy, negation, path, and ranking examples are dominated by semantically wrong executable queries or execution failures.

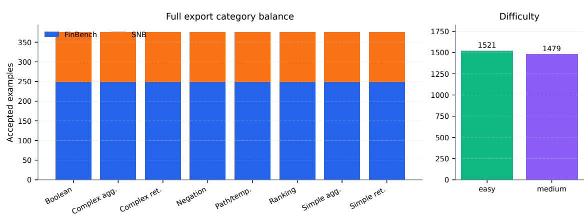

Figure 11: Full 3,000-example export distribution. The benchmark preserves the planned 2:1 FinBench/SNB graph mix while balancing all eight Cypher workload categories and maintaining a near-even easy/medium difficulty split.

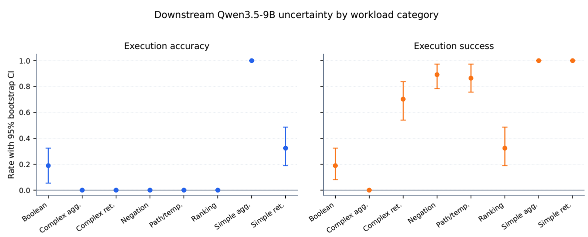

Figure 12: Bootstrap uncertainty for downstream Qwen3.5-9B Text2Cypher evaluation by workload category. Error bars show 95% row-level bootstrap intervals over the full exported test split.

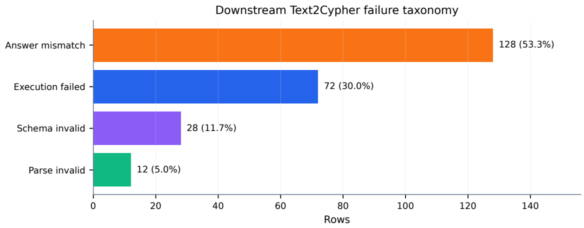

Figure 13: Downstream Text2Cypher failure taxonomy for the local Qwen3.5-9B baseline on the full test split. Exact matches are excluded so the chart exposes whether misses come from invalid Cypher, failed execution, or semantically wrong executable queries.

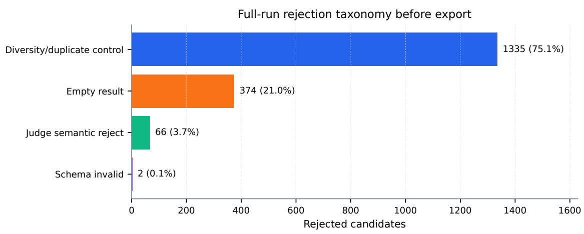

Figure 14: Full-run rejection taxonomy before benchmark export. Most rejected candidates come from duplicate/diversity control during category recovery and from empty execution results; schema-invalid Cypher is rare after deterministic governance.